GSS 的形式化定义：基于文本预测与认知不完备性的原子级工具调用数学模型

A Formal Definition of GSS: A Mathematical Model of Atomic Tool Invocation Founded on Text Prediction and Cognitive Incompleteness

Name: Constant-Dalink
Location: Hubei, China
Contact: 13581432693@163.com / 13581432693
Independent Researcher

Preprint v1.0 — 2026

摘要

当前主流大语言模型工具调用框架主要沿两条路径发展：一类依赖 JSON Schema 约束输出结构，另一类依赖视觉多模态模型处理屏幕截图。两类方法都可以工作，但在操作系统控制场景中，通常伴随较高的上下文负担、较长的输出链，以及额外的解析成本。

本文提出 **GSS（General Skills Standard）**，一种面向操作系统控制的文本原生原子指令标准。本文的出发点有两个。第一，大语言模型的基本工作方式是自回归的下一词预测，因此短文本动作序列通常比复杂结构化输出更接近其稳定生成区域。第二，任何有限认知系统在执行具体任务时，真正被激活的知识通常只是全部知识中的一个很小子集。本文以 **Constant-Dalink 不完备公式**

$$\forall \pm \{(V_1, V_2)^c, V_1, V_2, V, \dots\} \xrightarrow{\text{Check}} \circ V_{use}$$

对这一原则进行形式化刻画，并将其作为 GSS 的理论基础之一。

在此基础上，本文完成以下工作：(1) 将 Constant-Dalink 公式作为公理性基础引入，并与图灵完备性构成对偶分析框架；(2) 使用集合论与 BNF 文法定义 GSS 的 16 条原子指令及其组合形式；(3) 建立预测–调用等价性（PIE）的概率模型，分析文本原生调用相对 JSON 强制解析的可靠性优势；(4) 给出 Token 成本的渐近模型，说明 GSS 的上下文开销满足 $O(1/N)$ 压缩特征；(5) 说明 $C_{GSS} \approx \kappa_{Dalink}$，即 GSS 约 150 Token 的运行上下文与 Constant-Dalink 常数保持一致；(6) 讨论 16 条指令在操作系统控制域中的功能完备性与工程最小性。

本文的目标不是提出一个适用于所有任务的统一工具框架，而是给出一种更适合小模型、短上下文和操作系统控制任务的形式化表示。

关键词：工具调用；大语言模型；形式文法；Token 经济学；认知不完备性；Constant-Dalink 公式；边缘 AI

1. 引言

1.1. 能力上界与认知负载下界

1936 年，图灵通过通用图灵机刻画了可计算性的上界：如果一个系统能够模拟通用图灵机，那么它可以计算一切可计算函数。图灵理论关注的问题是：**一个形式系统最多能做什么。**

在智能系统的实际运行中，另一个问题同样关键：**一个系统在某一时刻真正需要激活多少知识。**

这一问题由 Constant-Dalink 不完备公式给出形式化回答：

$$\forall \pm \{(V_1, V_2)^c, V_1, V_2, V, \dots\} \xrightarrow{\text{Check}} \circ V_{use}$$

其核心结论是：在任意有限认知系统中，任务执行时真正进入运行态的知识集合 V_{use}，始终只是知识空间中的一个极小子集：

$$|V_{use}| \leq \kappa \ll |\mathcal{V}|$$

因此，图灵完备性与 Constant-Dalink 不完备性分别回答了两个层面的问题，如表 1 所示。

表 1: 图灵完备性与 Constant-Dalink 不完备性的比较

维度	图灵完备性	Constant-Dalink 不完备性
关注点	计算能力的上界	认知负载的下界
问题	最多能做什么	最少需要激活什么
工程含义	追求能力覆盖	追求按需加载
系统设计目标	能力可扩展	上下文可压缩

本文将这两条法则视为同一系统设计中的两个边界条件：一方面，系统需要具备扩展能力；另一方面，系统运行时的上下文负载应保持受控。

1.2. 当前工具调用框架的两个问题

现有大语言模型工具调用框架在操作系统控制场景中，通常存在两个共性问题。

第一，**输出形式与模型原生能力之间存在偏差。**大语言模型擅长条件文本生成，而 JSON 工具调用要求模型稳定输出括号、键名、层级、引号和字段结构。这种方式在接口上是清晰的，但并不总是对应模型最稳定的生成区域。

第二，**上下文组织方式与最小使用集原则不一致。**在许多框架中，每次调用都需要带入完整工具定义、全部历史消息，或完整屏幕截图。这意味着系统持续将接近全集 $\mathcal{V}$ 的信息推入上下文，而不是仅保留当前步骤所需的 V_{use}。

以操作系统控制为例，可以粗略比较几类方案，如表 2 所示。

表 2: 不同框架在能力扩展与上下文组织上的特点

框架	能力扩展	上下文组织
OpenAI Function Calling	工具集预定义	全量 Schema
LangChain Agent	工具集预定义	工具描述 + 历史消息
AutoGPT	部分代码执行	历史累积明显
Anthropic Computer Use	无显式自扩展	全屏截图输入
GSS-OS	HOW-code 可扩展	极小文本上下文

1.3. 本文目标

本文的目标是对一种更轻量的操作系统控制标准进行形式化定义。具体包括：

- 用少量原子指令描述操作系统控制动作；
- 让工具调用尽量回到 LLM 更稳定的短文本生成形式；
- 用 Constant-Dalink 公式解释按需加载、极简上下文与动态分库机制；
- 给出 GSS 的形式定义、概率解释与 Token 成本分析。

1.4. 文章结构

本文结构如下：

- 第 2 节介绍 Constant-Dalink 公式及其符号定义；
- 第 3 节回顾相关工作；
- 第 4 节给出 GSS 的形式化定义；
- 第 5 节建立预测–调用等价性（PIE）的概率模型；
- 第 6 节讨论 Token 成本；
- 第 7 节讨论完备性与最小性；
- 第 8 节给出公式到系统的映射；
- 第 9 节总结局限与结论。

2. Constant-Dalink 不完备公式

本节对 Constant-Dalink 公式进行逐符号定义，并作为后文推导的理论基础。

2.1. 公式

$$\forall \pm \{(V_1, V_2)^c, V_1, V_2, V, \dots\} \xrightarrow{\text{Check}} \circ V_{use}$$

该公式描述了一个从知识全集到最小使用集的压缩过程。

2.2. 全域二元量化 $\forall\pm$

定义 2.1 ($\forall\pm$). *设全体信息宇宙为 $\mathcal{U}$。$\forall\pm$ 表示系统面对的不是单纯的元素存在，而是每个元素的两种状态：加载（$+$）与未加载（$-$）。*

可形式化写为：

$$\forall\pm \equiv \forall\, S \in \mathscr{P}(\mathcal{U})$$

其中 $\mathscr{P}(\mathcal{U})$ 为 $\mathcal{U}$ 的幂集。

该定义的含义是：认知系统不仅要决定”使用什么”，还要决定”此刻不带什么”。在 GSS 中，`ld lib_web` 表示将某个库置于加载态，而其他未加载库保持在未激活状态。

2.3. 变量复合体 $\{(V_1, V_2)^c, V_1, V_2, V, \dots\}$

定义 2.2 (知识空间 $\mathcal{V}$). *设知识空间为：*

$$\mathcal{V} = \mathcal{V}_{atom} \cup \mathcal{V}_{comp} \cup \mathcal{V}_{ext}$$

其中：

$$\begin{aligned}
\mathcal{V}_{atom} &= \{V_1, V_2, \dots, V_n\} \\
\mathcal{V}_{comp} &= \{(V_i, V_j)^c \mid V_i, V_j \in \mathcal{V}\} \\
\mathcal{V}_{ext} &= \{V \mid V \text{ 尚未被创造}\}
\end{aligned}$$

其中：

- $\mathcal{V}_{atom}$ 表示不可再分的最小知识单元；
- $\mathcal{V}_{comp}$ 表示多个知识单元的有序组合；
- $\mathcal{V}_{ext}$ 表示尚未显式实现、但理论上可扩展得到的新能力。

在 GSS 中，$\mathcal{V}_{atom}$ 可对应 16 条原子指令，$\mathcal{V}_{comp}$ 对应指令序列，$\mathcal{V}_{ext}$ 对应未来可由 `hc` 生成的新工具。

2.4. Check 算子

定义 2.3 (Check). $\xrightarrow{Check}$ 表示从候选知识集合到可执行知识集合的压缩过程。本文将其分为三个阶段：

$$Check = Activate \circ Validate \circ Filter$$

三阶段的对应关系见表 3。

表 3: Check 算子的三阶段结构

阶段	数学作用	GSS 中的对应
Filter	保留与当前任务相关的候选项	解析器过滤无关或非法输出
Validate	检查格式与安全约束	白名单、参数校验、审查机制
Activate	将合法项映射为执行动作	执行器调用实际动作

形式化地，Check 可视为一个压缩映射：

$$\text{Check} : \mathscr{P}(\mathcal{V}) \to \mathcal{V}_{use}$$

并满足：

$$|\text{Check}(S)| \ll |S| \quad \forall S \in \mathscr{P}(\mathcal{V})$$

2.5. 激活算子 $\circ$ 与使用集 V_{use}

定义 2.4 (使用集 V_{use}). V_{use} 表示在具体任务中被实际激活的最小知识子集：

$$V_{use} \subseteq \mathcal{V}, \quad |V_{use}| = \min\{|S| : S \subseteq \mathcal{V},\ S \text{ 足以完成任务} \tau\}$$

符号 $\circ$ 表示激活或实例化。

也就是说，知识可以存在于系统中，但只有进入 $\circ V_{use}$ 的部分才参与当前执行。

2.6. 认知不完备性定理

定理 2.1 (Constant-Dalink 认知不完备性). 对于任何有限处理能力的认知系统 $\mathcal{C}$，在任意时刻 t 执行任务 τ 时，都有：

$$|V_{use}(\tau, t)| \leq \kappa \ll |\mathcal{V}|$$

其中 κ 是与系统运行结构相关的常数。

该定理的工程含义是：当系统在每步操作中都带入全部工具定义、全部历史和全部环境信息时，其上下文规模通常明显大于当前真正需要的 V_{use}。

推论 2.1 (全量加载的冗余性). *若系统在每次操作时加载全部知识，则冗余率为：*

$$Redundancy = 1 - \frac{|V_{use}|}{|\mathcal{V}|}$$

当 $|\mathcal{V}|$ 增长时，该冗余率趋近于 1。

3. 相关工作

3.1. 基于 JSON Schema 的工具调用

OpenAI 于 2023 年提出 Function Calling [1]，允许模型根据函数签名输出 JSON 调用。LangChain [2]、AutoGPT [3] 等系统在此基础上构建了不同形式的 Agent 工作流。

这一路线的优点是接口清晰、后处理方便；不足在于：

- 输出目标包含较多结构性 Token；
- Schema 自身会占用明显上下文；
- 工具数量增加时，提示长度随之增长。

3.2. 基于视觉输入的桌面控制

Anthropic 的 Computer Use [5] 展示了基于截图输入进行桌面控制的能力。这种方法保留了较完整的环境信息，但通常需要较高的 Token 成本和较强的视觉模型能力。

从 Constant-Dalink 的角度看，整张截图中的大部分像素并不属于当前动作所需的最小使用集。

3.3. GSS 的定位

GSS 采用第三种路径：**文本原生调用（Text-Native Invocation）**。其目标不是替代所有结构化接口或视觉系统，而是在操作系统控制这一任务域内，提供一种更轻量的动作表达方式。

表 4: 几类工具调用表示方式的比较

方案	表达载体	单步上下文成本	对模型的主要要求
JSON Schema	结构化文本	中高	结构输出稳定性
Visual Agent	截图 + 文本	高	多模态理解
GSS	原子动作文本	低	短文本动作映射

4. GSS 的形式化定义

4.1. 字母表与操作码集

定义 4.1 (字母表). 设 Σ 为 *ASCII* 可打印字符集，Σ^* 为其 *Kleene* 闭包。

定义 4.2 (GSS 操作码集).

$$\mathcal{O} = \{\mathtt{c}, \mathtt{dc}, \mathtt{rc}, \mathtt{sc}, \mathtt{dr}, \mathtt{hk}, \mathtt{t}, \mathtt{o}, \mathtt{k}, \mathtt{w}, \mathtt{sl}, \mathtt{ld}, \mathtt{hc}, \mathtt{mk_lib}, \mathtt{mv}, \mathtt{msg}\}$$

且

$$|\mathcal{O}| = 16$$

4.2. 操作码语义映射

定义语义映射：

$$\sigma : \mathcal{O} \to \mathcal{D}$$

其中 $\mathcal{D}$ 为动作语义域。对应关系如表 5 所示。

表 5: GSS 操作码语义

操作码	语义	域
c	左键单击	鼠标
dc	双击	鼠标
rc	右键单击	鼠标
sc	滚轮	鼠标
dr	拖拽	鼠标
hk	快捷键	键盘
t	输入文本	键盘
o	启动程序	进程
k	终止进程	进程
w	切换窗口	窗口
sl	等待	流控
ld	加载库	知识管理
hc	造器	扩展
mk_lib	建库	扩展
mv	移动工具	扩展
msg	消息	通信

4.3. 参数空间

为各操作码定义参数域：

$$
\begin{aligned}
\mathcal{A}_{\mathtt{c}} &= \mathcal{A}_{\mathtt{dc}} = \mathcal{A}_{\mathtt{rc}} = \mathcal{G} \\
\mathcal{A}_{\mathtt{dr}} &= \mathcal{G} \times \mathcal{G} \\
\mathcal{A}_{\mathtt{hk}} &= \mathcal{K}^{+} \\
\mathcal{A}_{\mathtt{t}} &= \Sigma^{+} \\
\mathcal{A}_{\mathtt{sl}} &= \mathbb{R}^{+} \\
\mathcal{A}_{\mathtt{o}} &= \mathcal{A}_{\mathtt{k}} = \mathcal{P}
\end{aligned}
$$

其中：

- $\mathcal{G}$：网格坐标集；
- $\mathcal{K}^{+}$：非空按键序列；
- $\mathcal{P}$：程序标识符集。

4.4. 指令与程序

定义 4.3 (GSS 指令).

$$\iota = (op, \alpha), \quad op \in \mathcal{O},\ \alpha \in \mathcal{A}_{op}$$

定义 4.4 (GSS 程序).

$$\Pi = \langle \iota_1, \iota_2, \ldots, \iota_n \rangle \in \mathcal{I}^{*}, \quad n \in \mathbb{N}^{+}$$

其中 $\mathcal{I}$ 为所有合法指令的集合。

在这一表示下，单条指令对应原子变量，指令序列对应组合变量。

4.5. 网格坐标空间

设屏幕分辨率为 $W \times H$，层级 $\ell \in \{1, \ldots, 8\}$。第 ℓ 层将屏幕划分为 $2^{\ell} \times 2^{\ell}$ 个网格。定义坐标到像素中心的映射：

$$\phi_{\ell}(c_j, r_i) = \left(\left\lfloor \frac{(2j-1)W}{2^{\ell+1}} \right\rfloor, \left\lfloor \frac{(2i-1)H}{2^{\ell+1}} \right\rfloor \right)$$

该映射将连续像素空间压缩为离散动作标号，因此也可视为空间维度上的 Check：

$$\Psi_{grid} : [0, W) \times [0, H) \xrightarrow{\text{Check}} \mathcal{G}_{\ell}$$

4.6. BNF 形式文法

```
<program>      ::= <line> { NEWLINE <line> }
<line>         ::= <instruction> | EMPTY

<instruction>  ::= <mouse_cmd> | <key_cmd> | <proc_cmd>
                 | <flow_cmd> | <know_cmd> | <evo_cmd> | <comm_cmd>

<mouse_cmd>    ::= "c" SP <grid>
                 | "dc" SP <grid>
                 | "rc" SP <grid>
                 | "sc" SP <dir> SP <int>
                 | "dr" SP <grid> SP <grid>

<key_cmd>      ::= "hk" SP <combo>
                 | "t" SP <text>

<proc_cmd>     ::= "o" SP <path>
                 | "k" SP <name>
                 | "w" SP <title>

<flow_cmd>     ::= "sl" SP <float>
<know_cmd>     ::= "ld" SP <lib_id>
<evo_cmd>      ::= "hc" SP <name>
                 | "mk_lib" SP <name>
                 | "mv" SP <name> SP <name>
<comm_cmd>     ::= "msg" SP <text>

<grid>         ::= LETTER+ DIGIT+
<combo>        ::= <key> { "+" <key> }
```

命题 4.1. *在给定词法约束下，GSS 指令语言可由有限状态机制完成基本识别，因此其解析复杂度可保持在线性时间范围内。*

5. 预测–调用等价性（PIE）的概率模型

本节采用简化模型，用于说明文本原生调用在长度和上下文方面的潜在优势。

5.1. 调用可靠性

设自回归语言模型为 $\mathcal{M}$，其条件概率为：

$$P_{\mathcal{M}}(t_{n+1} \mid t_{1:n})$$

对调用串 $\mathbf{s} = (s_1, \ldots, s_L)$，定义其可靠性为：

$$R(\mathbf{s} \mid \mathbf{ctx}) = \prod_{i=1}^{L} P_{\mathcal{M}}(s_i \mid \mathbf{ctx}, s_{1:i-1})$$

这是一种理想化写法，它将整体调用成功近似为逐 Token 成功的乘积。

5.2. 长度因素

若以平均 Token 正确率 $\bar{p}$ 近似，则：

$$R \approx \bar{p}^L$$

设 GSS 调用长度为 L_G，JSON 调用长度为 L_J，则：

$$\Gamma_{length} = \frac{\bar{p}^{L_G}}{\bar{p}^{L_J}} = \bar{p}^{L_G - L_J}$$

例如，当 $\bar{p} = 0.95$，$L_G = 3$，$L_J = 20$ 时：

$$\Gamma_{length} = 0.95^{-17} \approx 2.43$$

这说明在相同平均正确率假设下，较短输出更有利。

5.3. 上下文因素

实际系统中，$\bar{p}$ 会受到上下文长度、结构复杂度与提示干扰的影响。如果系统要求模型同时处理大量 Schema、历史消息和嵌套结构，那么模型用于” 当前动作决策” 的有效注意力会被分散。

因此，可写成更一般的形式：

$$\Gamma = \frac{\bar{p}_G^{L_G}}{\bar{p}_J^{L_J}}$$

其中通常期待满足：

$$L_G < L_J, \qquad \bar{p}_G \geq \bar{p}_J$$

5.4. PIE 的定义

定义 5.1 (预测–调用等价性，PIE). *若对目标动作集合 $\mathcal{A}_{target}$ 中任意动作 a，存在某个可生成字符串 $\mathbf{s}$，满足：*

$$P_{\mathcal{M}}(\mathbf{s} \mid \mathbf{ctx}) > \theta \quad \text{且} \quad \mathcal{E}(\mathbf{s}) = a$$

则称系统满足预测–调用等价性（PIE）。

这里 $\mathcal{E}$ 表示将文本调用解析并执行的函数。

5.5. 门槛比较

若继续采用平均正确率近似，达到成功阈值 θ 的充分条件为：

$$\bar{p} > \theta^{1/L}$$

因此：

$$\bar{p}_G > \theta^{1/L_G}, \qquad \bar{p}_J > \theta^{1/L_J}$$

取 $\theta = 0.8$，$L_G = 3$，$L_J = 20$，则：

$$\bar{p}_G > 0.928, \qquad \bar{p}_J > 0.989$$

这说明较长结构输出对单 Token 准确率提出了更高要求，这也是小模型更适合短文本原子调用的重要原因之一。

6. Token 成本模型

6.1. 传统框架

将传统工具调用框架的单轮成本记为：

$$C_{trad} = S_{schema} + S_{state} + S_{output} + S_{hist}$$

其中：

- S_{schema}：工具定义或 Schema；
- S_{state}：当前状态描述；
- S_{output}：模型输出；
- S_{hist}：历史消息。

当工具数量、状态维度或对话轮数增加时，C_{trad} 往往随之增长。

6.2. GSS 框架

GSS 的单轮成本可写为：

$$C_{GSS} = S_{dict} + S_{grid} + S_{lib} + S_{task} + S_{cmds}$$

其中：

- S_{dict}：16 条指令的简短定义；
- S_{grid}：当前网格层级；
- S_{lib}：已加载库列表；
- S_{task}：当前任务说明；
- S_{cmds}：输出的动作序列。

一个典型近似是：

$$C_{GSS} \approx 80 + 8 + 15 + 20 + 5n$$

对一个 5 步任务，有：

$$C_{GSS} \approx 148 \text{ Token}$$

6.3. Dalink 常数等式

定理 6.1 (Dalink 常数等式). *在 GSS 的运行设计中，其上下文成本与 Constant-Dalink 常数保持一致：*

$$C_{GSS} \approx \kappa_{Dalink} \approx 150 \ \textit{Token}$$

这里的 κ_{Dalink} 表示在当前任务域与接口设计下的经验常数量级。

6.4. 压缩比

定义压缩比：

$$\rho = \frac{C_{GSS}}{C_{trad}}$$

若传统框架的成本随工具数量、历史长度和状态复杂度增长，而 GSS 在局部任务步骤中保持近似常数级，则在一定条件下有：

$$\rho = O\left(\frac{1}{N}\right)$$

其中 N 表示传统框架中随规模增长的结构因子。

这意味着：随着工具和历史规模增长，轻量文本调用在成本上可能具备越来越明显的优势。

7. 完备性、最小性与扩展性

7.1. 核心交互覆盖

定义核心交互集合：

$$\mathcal{O}_{core} = \{\mathtt{c}, \mathtt{dc}, \mathtt{rc}, \mathtt{sc}, \mathtt{dr}, \mathtt{hk}, \mathtt{t}, \mathtt{o}, \mathtt{k}, \mathtt{w}\}$$

这 10 条指令覆盖了常见的人机操作通道，如表 6 所示。

表 6: 人机交互通道与 GSS 覆盖

交互通道	GSS 对应
鼠标点击 / 双击 / 右击 / 滚动 / 拖拽	c, dc, rc, sc, dr
键盘快捷键与文本输入	hk, t
程序打开 / 关闭 / 切换	o, k, w

7.2. 不可完全替代性

这些动作之间并不总能无损互相替代。例如：

- dc 不能简单视为两个普通 c，因为双击依赖时间窗口；
- hk 不等同于 t，前者触发的是组合键事件；
- dr 不等同于两次点击，因为拖拽要求持续按压。

因此，它们保留为独立原子指令具有工程合理性。

7.3. 扩展能力

GSS 并不将 16 条指令视为封闭能力全集。通过 ld、hc、mk_lib、mv 等机制，系统可以在原子交互层之上获得更高层工具能力。

因此，更准确的说法是：

- 底层动作空间是小而固定的；
- 高层能力空间是可扩展的。

7.4. 最小性讨论

16 条指令可以分解为：

1. 核心桌面交互：10 条；
2. 时序控制：1 条（`sl`）；
3. 动态加载：1 条（`ld`）；
4. 扩展机制：3 条（`hc`, `mk_lib`, `mv`）；
5. 消息输出：1 条（`msg`）。

因此：

$$10 + 1 + 1 + 3 + 1 = 16$$

这里的”最小性”更适合理解为工程最小性：在不明显牺牲表达能力的前提下，尽量减少常驻指令数量。

7.5. 对偶完备性

定理 7.1 (GSS 的对偶完备性). *GSS-OS 同时满足：*

$$\begin{cases} \textbf{图灵上界：} & \mathcal{O}_{core} \cup \boldsymbol{hc} \to \textit{Python} \text{ 可模拟任意图灵机} \\ \textbf{\textit{Dalink} 下界：} & |V_{use}| \leq \kappa, \quad C_{GSS} \approx 150 \ \textit{Token} \end{cases}$$

这意味着，GSS 既保留了扩展能力的上界，又对运行时的认知负载施加了常数级约束。

8. 公式到系统的映射

将 Constant-Dalink 公式映射到 GSS-OS，可得到表 7 的对应关系。

表 7: Constant-Dalink 公式与 GSS 子系统映射

公式组件	含义	GSS 子系统	工程文件
$\forall\pm$	加载 / 不加载决策	动态分库	`lib_manager.py`
$(V_1, V_2)^c$	组合知识	指令序列	`executor.py`
V_1, V_2	原子知识单元	16 条指令	`prompt.py`
V	一般知识项	动态库工具	`libs/*.yaml`
...	开放知识空间	HOW-code	`howcode.py`
Check	过滤–校验–激活	parser + gate	`parser.py`, `gate.py`
$\circ$	激活与实例化	执行器	`executor.py`
V_{use}	当前最小使用集	最终 Prompt	`prompt.py`
κ	常数级预算	系统约束	`config.py`

9. 局限性与结论

9.1. 局限性

本文仍有若干局限：

1. **关于 Constant-Dalink 公式的跨域标定：**本文主要在操作系统控制域中讨论 κ 的经验量级，更大范围的跨任务标定仍需进一步实验。
2. **关于 PIE 模型的简化：**第 5 节采用平均 Token 正确率近似，忽略了位置依赖、解码策略与执行器容错等因素。
3. **关于任务域边界：**GSS 的优势主要体现在桌面控制、短动作序列和有限上下文预算场景。对于高度依赖复杂视觉或长结构化数据的任务，其他框架仍可能更适合。
4. **关于安全性：**尽管原子指令与白名单有助于提高可控性，但只要系统具备扩展能力，就仍然需要严格的人类审查、权限隔离和执行沙箱。

9.2. 结论

本文提出了 GSS，一种面向操作系统控制的文本原生原子调用模型，并给出了其形式化描述。

全文的核心观点可以概括为两点：

1. 对于工具调用而言，短文本动作序列更接近 LLM 的基本生成机制；
2. 对于任务执行而言，实际需要激活的知识通常远小于知识全集，因此上下文应尽量围绕最小使用集构建。

在这一思路下，本文：

- 给出了 Constant-Dalink 公式的逐符号定义；
- 用集合论和 BNF 文法形式化了 GSS；
- 用简化模型讨论了文本调用在可靠性与 Token 成本上的优势；
- 讨论了 GSS 在桌面交互、最小性与可扩展性之间的平衡。

更准确地说，GSS 不是对所有工具调用范式的替代，而是一种适用于**低上下文、强动作化、可审查执行**场景的工程标准。它关注的问题不是系统理论上能携带多少信息，而是：**在当前这一步中，真正需要进入执行面的是什么。**

参考文献

[1] OpenAI. Function Calling and Other API Updates. *OpenAI Blog*, 2023.

[2] Harrison Chase. LangChain: Building Applications with LLMs through Composability. *GitHub*, 2022.

[3] Significant Gravitas. AutoGPT: An Autonomous GPT-4 Experiment. *GitHub*, 2023.

[4] Nelson F. Liu et al. Lost in the Middle: How Language Models Use Long Contexts. *TACL*, 2024.

[5] Anthropic. Developing a Computer Use Model. *Anthropic Research*, 2024.

[6] Tom B. Brown et al. Language Models are Few-Shot Learners. *NeurIPS*, 2020.

[7] Jason Wei et al. Chain-of-Thought Prompting Elicits Reasoning in Language Models. *NeurIPS*, 2022.

[8] Timo Schick et al. Toolformer: Language Models Can Teach Themselves to Use Tools. *NeurIPS*, 2023.

[9] Shunyu Yao et al. ReAct: Synergizing Reasoning and Acting in Language Models. *ICLR*, 2023.

[10] Claude E. Shannon. A Mathematical Theory of Communication. *Bell System Technical Journal*, 1948.

[11] Naftali Tishby et al. The Information Bottleneck Method. *Allerton Conference*, 1999.

[12] Kurt Gödel. Über formal unentscheidbare Sätze. *Monatshefte für Mathematik*, 1931.

[13] Alan M. Turing. On Computable Numbers. *Proceedings of the London Mathematical Society*, 1936.

[14] GSS-OS Research Group. Constant-Dalink Incompleteness Formula. *Internal Notes*, 2025.